

Cody Researcher - Design Note

Engineering decisions behind the autonomous web research agent

1. The problem, in one paragraph

Given a company and a plain-English question, find the official site, work out which pages actually matter, read them, and answer with real citations - without hard-coding for any one website, without crawling blindly forever, and without ever saying something the pages don't actually support. The interesting engineering problem isn't calling a browser tool; it's deciding when to stop, what to trust, and how to keep it fast and cheap at the same time.

2. Pipeline shape

The agent runs as a small state graph rather than one long function: **resolve -> seed -> crawl -> decide -> synthesize -> validate -> finalize**. Resolve turns whatever the user typed into a starting URL and an allowed domain. Seed reads the homepage and also fires one search query built from the task itself, which usually finds the right deep page in one hop instead of several clicks. Crawl fetches a small batch of queued pages at a time, pulls facts out of each with an LLM call, and queues any new links that pass a relevance check. Decide is a plain-code checkpoint: keep going, or stop, based on the queue and the guardrails. Synthesize writes the final answer with citation markers from the evidence collected so far. Validate is plain code again - it checks every citation marker actually points at something real before the answer goes out.

Batching the crawl (rather than crawling everything, then asking one giant question at the end) matters for two reasons: it lets the agent stop as soon as it has enough, and it keeps the decide step running often enough to actually enforce the guardrails instead of only checking them once at the very end.

3. Where the LLM is used, and where it deliberately isn't

Three model calls do real reasoning work: choosing the official site out of noisy search results, deciding which links are worth following for this specific task, and pulling facts out of a page's text. A fourth writes the final answer from the collected evidence. Everything else - stripping HTML down to readable text, extracting links, canonicalizing and deduplicating URLs, retrying a failed fetch, checking whether a citation marker is real - is plain code. None of that needs judgment, and running it through a model would only add latency, cost, and a new way for things to go subtly wrong. Roughly one LLM call happens per page fetched, plus one per batch of newly discovered links - not one per link and not one per sentence.

4. Tool choices

Tool	Used for	Why this one
Groq (openai/gpt-oss-120B)	Site resolution, link relevance, evidence extraction, final answer	Judgment calls that don't reduce to rules
Playwright	JS-heavy pages, as a fallback	Required by the brief; only used when a plain HTTP GET fails or looks b

httpx	The default page fetch	Far cheaper than a browser; most pages don't need one
trafilatura	HTML to readable text	Boilerplate removal is a solved, deterministic problem
LangGraph	Orchestration	Explicit loop-until-done shape, and its SQLite checkpointer gives resumability
React + Tailwind	Frontend	Several panels driven by shared run state; Vite build served as static file
SQLite	All storage	Single-process local tool; no server to run, WAL mode is enough concurrency
Tavily / DuckDuckGo	Web search	Tavily if a key is set (built for agents); DuckDuckGo HTML scrape as a fallback

Firecrawl was deliberately left out - it mostly wraps the same two things this project already does directly (a headless browser plus boilerplate stripping), so it would add a dependency without adding a capability.

5. Guardrails

Every limit maps to a specific way an agent can go wrong if left unchecked: page count caps total browsing per task, iteration count catches loop bugs independent of page count, LLM-call count caps spend directly, wall-clock time catches slow sites, retry count bounds per-URL effort, and concurrency caps resource use. Defaults (40 pages, 15 iterations, 40 LLM calls, 10 minutes, 2 retries, 5 concurrent fetches) are set for a single research question on a normal company site - generous enough to actually find an answer, small enough that a bug can't turn into an unbounded bill or an unbounded crawl. All of them are environment variables, not constants, so they can be tuned per deployment without a code change.

6. Failure handling

Nothing fails silently. A bad LLM response gets one retry with a stricter prompt, then raises a typed error the caller must handle - a failed relevance check just means fewer links get queued that round, and a failed evidence extraction just means that one page contributes nothing. A failed page fetch is retried, then marked failed with its error saved, and the crawl moves on; the run's final status becomes *completed_with_failures* rather than pretending everything worked. If the process itself dies mid-run, the LangGraph SQLite checkpointer means the next resume call picks up from the last completed step instead of starting over.

7. Citation integrity

Evidence is tagged at the point it's extracted - *stated*, *extracted*, or *inferred* - and the synthesis prompt is told to keep that distinction visible in the answer, plus to say plainly when sources disagree instead of picking one silently. After the answer is written, plain code checks every citation marker in it against the evidence that was actually collected during that run; a marker that doesn't match anything real is recorded as invalid and shown as such, rather than trusted. Evidence itself always carries its source URL, fetch time, and fetch method, so any claim can be traced back to a specific page.

8. What changes at real production scale

- SQLite to Postgres for the app tables - the schema is already plain SQL, no ORM, so this is a connection-string change plus WAL-specific assumptions removed, not a rewrite.

- The single FastAPI background task becomes a proper queue (e.g. Postgres-backed job queue or Redis/RQ) so runs survive a process restart without relying on the checkpoint alone, and so multiple runs don't compete for one process's event loop.
- Per-domain rate limiting and a robots.txt check before crawling, which this version doesn't enforce yet.
- A second extraction pass for PDFs and other non-HTML documents linked from a site, since those are currently treated as fetch failures.
- Splitting the crawl-batch fetch/extract workers from the API process, so a slow research job can't starve the endpoints answering status polls.

9. Honest gaps in this version

- Testing against real, unfamiliar websites has to be run by whoever runs this project locally - see docs/test_report.md for why and how.
- No robots.txt or rate-limit awareness yet; MAX_CONCURRENCY is a blunt substitute for per-site politeness.
- Page content that lives only in a PDF or an image is not extracted in this version.
- The relevance filter and evidence extraction each run as one LLM call per batch/page; a single very large page is still read within one call's token budget, not chunked.